

PROMTS

Actúa como asistente de programación para una práctica de Estructura de Datos en Python. Contexto: Ya tengo un proyecto desarrollado en clase con Python, programación orientada a objetos y una interfaz gráfica creada con Qt Designer. En el proyecto ya existe una clase Stack con los métodos básicos de una pila. También existe una ventana o formulario donde se capturan datos y se muestran resultados, así como la clase que realiza la carga y la detección del evento para convertir la expression. Objetivo: Necesito agregar al proyecto existente la funcionalidad para convertir una expresión infija a expresión posfija utilizando la estructura de pila ya creada en clase. Necesito se genere una clase que contenga una pila y un método para llevar a cabo la conversión. Instrucciones importantes: 1. No generes un proyecto nuevo desde cero. 2. No reemplaces la clase Stack existente. Debes utilizarla e integrarte a su forma actual de trabajo. 3. Respeta la organización actual de archivos, clases, nombres de métodos y estilo de programación del proyecto. 4. Si necesitas proponer cambios, indícalos como cambios mínimos y explica exactamente en qué archivo y en qué parte deben colocarse. 5. La solución debe estar hecha con programación orientada a objetos. 6. La interfaz gráfica debe seguir usando el diseño generado con Qt Designer; no cambies el archivo .ui salvo que sea estrictamente necesario. 7. La aplicación debe permitir capturar una expresión infija, convertirla a posfija y mostrar el resultado en la interfaz. 8. Se asume que la expresión infija capturada es válida ya que de otra forma la conversión puede fallar. 9. Usa operadores básicos: +, -, *, /, \$(Potencia) y paréntesis. 10. Explica brevemente cómo funciona el algoritmo y cómo se utiliza la pila en la conversión, para esto genera un archivo .md que contenga esta explicación. Código existente:

```
from estructuras.lineales.lista_enlazada_simple import LinkedList class OperacionesPila: def __init__(self, lista: LinkedList): self.lista = lista self.resultado = "" def pila_agregar(self, data): self.lista.insert_at_beginning(data) self.resultado = f"Elemento {data} agregado a la pila." return self.resultado def pila_eliminar(self): eliminado = self.lista.delete_first() if eliminado is None: self.resultado = "La pila está vacía, no se puede eliminar." else: self.resultado = f"Elemento {eliminado} eliminado de la pila." return eliminado def pila_tope(self): if self.lista.head is None: self.resultado = "La pila está vacía." return None self.resultado = f"El tope de la pila es: {self.lista.head.data}" return self.lista.head.data def pila_imprimir(self): if self.lista.head is None: self.resultado = "La pila está vacía." return self.resultado elementos = [] temp = self.lista.head while temp is not None: elementos.append(str(temp.data)) temp = temp.next self.resultado = " -> ".join(elementos) return self.resultado def is_empty(self): return self.top is None import sys from PyQt5.QtWidgets import QApplication from load.load_MenuPrincipal import MenuPrincipal def main(): app = QApplication(sys.argv) ventana = MenuPrincipal() ventana.show() sys.exit(app.exec_()) if __name__ == "__main__": main() from PyQt5.QtWidgets import QDialog from PyQt5 import uic from menus.menu_pila import OperacionesPila from estructuras.lineales.lista_enlazada_simple import LinkedList class LoadInterfazPila(QDialog): def __init__(self): super().__init__() self.ui = uic.loadUi('ui/interfaz_pila.ui', self) # Crear la lista y el controlador de operaciones self.lista = LinkedList() self.operaciones = OperacionesPila(self.lista) # Conectar botones self.btn_pop.clicked.connect(self.pila_eliminar) self.btn_print.clicked.connect(self.pila_print) self.btn_push.clicked.connect(self.pila_agregar) self.btn_top.clicked.connect(self.pila_tope) def pila_agregar(self): dato = self.txt_dato.text() if dato.strip() == "": self.lbl_resultado.setText("Ingrese un dato válido.") return self.operaciones.pila_agregar(dato) self.lbl_resultado.setText(self.operaciones.resultado) def pila_eliminar(self): eliminado = self.operaciones.pila_eliminar() self.lbl_resultado.setText(self.operaciones.resultado) def pila_print(self): resultado = self.operaciones.pila_imprimir() self.lbl_resultado.setText(resultado) def pila_tope(self): tope = self.operaciones.pila_tope() self.lbl_resultado.setText(self.operaciones.resultado)
```

PROMTS

Resultado esperado de tu respuesta: Estructura de Datos | Práctica con IA Ing. Alejandro Montes - Primero analiza la estructura del código existente. - Después propón la clase con el método para convertir de infija a posfija. - Luego indica cómo conectar ese método con el botón o evento de la interfaz gráfica. - Finalmente proporciona pruebas para comprobar que funciona. - No omitas explicación, porque debo entender el código para defenderlo en clase ante el profesor y mis compañeros.

RESPUESTA DE LA IA:

📖 Estructura de Datos | Práctica con IA

****Profesor: Ing. Alejandro Montes****

Alejandro, vamos paso a paso para que tengas una solución clara, integrada y defendible en clase.

🔍 1. Análisis del código existente

- Ya tienes una ****clase `Stack`**** implementada con operaciones básicas (push, pop, top, imprimir).

- La interfaz gráfica (`LoadInterfazPila`) está conectada a botones que llaman métodos de la clase `OperacionesPila`.

- El flujo actual:

1. Capturas un dato en la interfaz.
2. Lo envías a la pila.
3. Muestras resultados en un `QLabel`.

👉 Esto significa que ****no debemos crear otra pila****, sino ****usar la existente**** para la conversión de expresiones.

🛠️ 2. Nueva clase para conversión infija → posfija

PROMTS

Crearemos una clase llamada `ConvertidorExpresiones` que **utiliza la pila existente** para aplicar el algoritmo de conversión (Shunting Yard simplificado).

📁 **Archivo sugerido:** `estructuras/convertidor\_expresiones.py`

```
```python
```

```
from estructuras.lineales.lista_enlazada_simple import LinkedList
```

```
from menus.menu_pila import OperacionesPila
```

```
class ConvertidorExpresiones:
```

```
 def __init__(self):
```

```
 # Usamos la pila ya implementada
```

```
 self.lista = LinkedList()
```

```
 self.pila = OperacionesPila(self.lista)
```

```
 def prioridad(self, operador):
```

```
 if operador == '$': # Potencia
```

```
 return 3
```

```
 if operador in ('*', '/')
```

```
 return 2
```

```
 if operador in ('+', '-')
```

```
 return 1
```

```
 return 0
```

```
 def infija_a_posfija(self, expresion):
```

```
 salida = []
```

```
 for token in expresion:
```

```
 if token.isalnum(): # Operando
```

```
 salida.append(token)
```

## PROMTS

```
elif token == '(':
 self.pila.pila_agregar(token)
elif token == ')':
 while self.lista.head and self.lista.head.data != '(':
 salida.append(self.pila.pila_eliminar())
 self.pila.pila_eliminar() # eliminar '('
else: # Operador
 while (self.lista.head and
 self.prioridad(self.lista.head.data) >= self.prioridad(token)):
 salida.append(self.pila.pila_eliminar())
 self.pila.pila_agregar(token)

Vaciar pila
while self.lista.head:
 salida.append(self.pila.pila_eliminar())

return " ".join(salida)
'''

```

### ## 3. Conexión con la interfaz gráfica

En tu clase `LoadInterfazPila`, agrega un **nuevo botón** en el `.ui`` (ejemplo: `btn_convertir``) y un `QLineEdit`` para capturar la expresión infija (`txt_expresion``).

 **Archivo:** `load/load_InterfazPila.py``

```
```python
```

```
from estructuras.convertidor_expresiones import ConvertidorExpresiones
```

PROMTS

```
class LoadInterfazPila(QDialog):
    def __init__(self):
        super().__init__()
        self.ui = uic.loadUi('ui/interfaz_pila.ui', self)

        self.lista = LinkedList()
        self.operaciones = OperacionesPila(self.lista)
        self.convertidor = ConvertidorExpresiones()

        # Conectar botones existentes
        self.btn_pop.clicked.connect(self.pila_eliminar)
        self.btn_print.clicked.connect(self.pila_print)
        self.btn_push.clicked.connect(self.pila_agregar)
        self.btn_top.clicked.connect(self.pila_tope)

        # Nuevo botón para conversión
        self.btn_convertir.clicked.connect(self.convertir_expresion)

    def convertir_expresion(self):
        expresion = self.txt_expresion.text()
        if expresion.strip() == "":
            self.lbl_resultado.setText("Ingrese una expresión válida.")
            return
        resultado = self.convertidor.infija_a_posfija(expresion)
        self.lbl_resultado.setText(f"Posfija: {resultado}")
    ...
```

⚠️ **Cambio mínimo en el `ui`:**

PROMTS

- Agregar un botón `btn\_convertir`.
- Agregar un `QLineEdit` llamado `txt\_expresion`.
- No se modifica nada más.

4. Pruebas de funcionamiento

Entrada:

...

A + B * C

...

Proceso:

1. `A` → salida.
2. `+` → pila.
3. `B` → salida.
4. `\*` → pila (mayor prioridad).
5. `C` → salida.
6. Vaciar pila → `\* +`.

Resultado esperado:

...

Posfija: A B C * +

...

Otro ejemplo:

...

(A + B) * C

...

Resultado:

...

Posfija: $A B + C *$

...

📄 5. Archivo de explicación (`docs/infija\_posfija.md`)

```markdown

# Conversión de Expresiones Infijas a Posfijas

## ¿Qué es una expresión infija?

Es la forma común de escribir operaciones:

...

$A + B * C$

...

## ¿Qué es una expresión posfija?

Es una notación donde los operadores van después de los operandos:

...

$A B C * +$

...

## Algoritmo (Shunting Yard simplificado)

1. Leer la expresión de izquierda a derecha.
2. Si es operando → enviar a la salida.
3. Si es operador → usar la pila para decidir cuándo sacarlo.
4. Si es paréntesis → manejar apertura y cierre con la pila.
5. Al final → vaciar la pila.

## PROMTS

### ## Uso de la pila

La pila se utiliza para **guardar operadores y paréntesis** mientras se procesa la expresión.

Esto asegura que los operadores se coloquen en el orden correcto según su **prioridad**.

Ejemplo:

'''

Infija: ( A + B ) \* C

Posfija: A B + C \*

'''

'''

---

✓ Con esto tienes:

- Una clase nueva integrada al proyecto.
- Conexión mínima con la interfaz.
- Ejemplos de prueba.
- Explicación en `.md` para defenderlo en clase.

CORRECCION QUE LE HICE A MI CODIGO:

MOSTRABA ERROR: Error: 'ConvertidorExpresiones' object has no attribute 'prioridad'

Y ME DIO EL SIGUIENTE CODIGO CORREGIDO:

```
class ConvertidorExpresiones:
```

```
 def __init__(self):
```

```
 from menus.menu_pila import OperacionesPila
```

```
 from estructuras.lineales.lista_enlazada_simple import LinkedList
```

```
 self.pila = OperacionesPila(LinkedList())
```



## PROMTS

```
def prioridad(self, operador):
 if operador == '$': # Potencia
 return 3

 if operador in ('*', '/'):
 return 2

 if operador in ('+', '-'):
 return 1

 return 0
```

```
def infija_a_posfija(self, expresion):
 salida = []

 for token in expresion:
 if token.isalnum():
 salida.append(token)

 elif token == '(':
 self.pila.pila_agregar(token)

 elif token == ')':
 while not self.pila.is_empty() and self.pila.pila_tope() != '(':
 salida.append(self.pila.pila_eliminar())
 self.pila.pila_eliminar()

 else: # Operador
 while (not self.pila.is_empty() and
 self.prioridad(self.pila.pila_tope()) >= self.prioridad(token)):
 salida.append(self.pila.pila_eliminar())
 self.pila.pila_eliminar()
 self.pila.pila_agregar(token)

 while not self.pila.is_empty():
 salida.append(self.pila.pila_eliminar())
```

PROMTS

```
return " ".join(salida)
```